Gauss–Legendre Replaces Driscoll–Healy in EquiformerV2: A Drop-in 31% Inference Speedup at Matched Equivariance

Anonymous Authors

Abstract

EquiformerV2 and other recent SO(3)-equivariant networks for atomistic modeling apply pointwise nonlinearities by mapping spherical-harmonic (SH) coefficients to a discrete grid on the sphere, applying a scalar activation, and integrating back via quadrature. The standard implementation, inherited from `e3nn`, uses an equiangular Driscoll–Healy (DH) grid. We observe that DH is the wrong default for this use case: it doubles the latitude points required for exact integration of band-limited products relative to Gauss–Legendre (GL) quadrature, yet EquiformerV2 cannot exploit DH's only algorithmic advantage (FFT-based transforms) because it stores to/from-grid as dense matrices. The result is an avoidable equivariance penalty. We implement a drop-in `CustomSO3Grid` that replaces DH with GL. On the fairchem default EquiformerV2 architecture (12 layers / 128 channels / $\ell_{\max}$=6), GL reaches the same kernel-level S^2-activation equivariance error as DH oversampled $2\times$ at $\mathbf{113.17 \pm 2.25}$ ms per forward—versus 164.66 ± 1.62 ms for DH $2\times$—a savings of $\mathbf{51.5 \pm 2.36}$ **ms** ($\mathbf{31.3}\%$, Welch's t-test $p < 10^{-4}$ on n=5 run means with distinct QM9 batches per iteration). The wall-clock savings come entirely from the S2-activation kernel, which grows from 8.5% to 39.1% of forward time when DH is oversampled, while everything else (attention, FFN, normalization) is unchanged. We further verify on four 50-epoch DH-trained QM9 checkpoints (a smaller 4-layer / $\ell_{\max}$=4 model, since publicly trained 12-layer fairchem checkpoints do not exist for this benchmark) that swapping in GL at inference, without any retraining, preserves predictions: test MAE shifts by ≤ 0.01 eV (well within seed-to-seed training variance); trained-model prediction-invariance error is unchanged (the model has learned to compensate for its training-time grid, so the *kernel-level* equivariance gain only translates to *trained-model* equivariance gain when GL is used at training time too). The headline operational claim is therefore: *GL is a drop-in replacement that delivers the wall-clock and kernel-equivariance benefits with no retraining or accuracy regression.* We position this against concurrent work that addresses the same equivariance problem via architectural change (EquiformerV3's SwiGLU-S^2, `arXiv:2604.09130`) or alternative sampling layouts (Equivariant Spherical Transformer, `arXiv:2505.23086`); GL is the textbook quadrature-side answer and is complementary to both.

1 Introduction

Spherical-harmonic neural networks for atomistic modeling. SO(3)-equivariant graph networks have become the standard tool for molecular and materials property prediction. The line of work from SCN Zitnick et al. [2022] to eSCN Passaro and Zitnick [2023] to EquiformerV2 Liao et al. [2024] represents per-node features as functions on the sphere, expanded in spherical harmonics (SH)

Preprint. Under review.

up to a band limit $\ell_{\max}$. The dominant nonlinearity in this design is the *S2 activation*: project SH coefficients onto a finite grid on S^2, apply a scalar activation pointwise, integrate back via quadrature. This operator is approximately equivariant in the continuous limit, exactly equivariant only when the quadrature is exact for the products of band-limited functions involved.

The known equivariance gap and the community response. The original EquiformerV2 paper acknowledges that "sampling-induced equivariance error remains an open challenge" Liao et al. [2024]. The `fairchem` community has filed and discussed this issue (`FAIR-Chem/fairchem#1068`, March 2025), where users observe that equivariance error of order $\sim 10^{-2}$ is reached at the default grid, dropping only to $\sim 10^{-4}$ at 1024^2 grid points fairchem community [2025]—a prohibitive cost. Two very recent papers attack the same problem with different tools. *EquiformerV3* Liao et al. [2026] (arXiv 2604.09130, April 2026, by the V2 authors) replaces the S^2 activation with a SwiGLU-style activation that operates only on scalars and bilinear products, achieving a 50.6% reduction in grid points while maintaining strict equivariance, but as an *architectural* change requiring retraining. *Equivariant Spherical Transformer (EST)* An et al. [2025] (arXiv 2505.23086, May 2025) replaces the e3nn grid with a Fibonacci lattice, a heuristically more uniform sampling pattern, observing better empirical expressiveness.

The choice nobody has made: a different quadrature. What none of these works alters is the choice of *quadrature rule*. EquiformerV2's `SO3_Grid` delegates to e3nn's `ToS2Grid` / `FromS2Grid`, which use an equiangular Driscoll–Healy (DH) grid with Kostelec–Rockmore weights. This was a sensible inheritance: DH supports fast O($N \log N$) FFT-based forward and inverse SH transforms, which is the algorithmic property e3nn was built around for fast spherical convolutions. But EquiformerV2 does not use the FFT—it precomputes `to_grid_mat` and `from_grid_mat` as dense matrices and applies them via `einsum`. Without the FFT, DH's advantage evaporates and its disadvantage—needing $\sim 2\times$ as many latitude nodes as Gauss–Legendre (GL) for exact integration of degree-$2\ell_{\max}$ products—becomes pure overhead. The classical numerical-analysis result that *GL needs roughly half the latitude points of DH for matched accuracy on the sphere* Wieczorek and Meschede [2018], Hirt et al. [2015] is more than a century old, and is the standard textbook fact about spherical quadrature. It does not appear to have been applied to the EquiformerV2 line of work.

This paper. We make this swap and measure what happens at three levels. (i) *Kernel level*: equivariance error of a single S^2-activation under each grid (Sec. "Kernel-level equivariance", weight-independent; GL match-DH and DH $2\times$ both reach the truncation floor while DH default and GL min stay above it). (ii) *Architecture level*: end-to-end forward wall-clock under each grid using a freshly-instantiated 12-layer EquiformerV2 backbone, with all measurements on *independently-sampled* QM9 batches (no batch reuse within a run) and all confidence intervals computed over n=5 run means with a t-distribution at df=4. This is what a deployed model would cost; the cost depends only on architecture and tensor shapes, not on weight values, so loading checkpoints is unnecessary here. (iii) *Drop-in on trained checkpoints*: load four actual 50-epoch QM9 checkpoints trained with DH default, swap in GL without any retraining or fine-tuning, and re-measure prediction invariance error and test MAE on the held-out test set. **Scope note**: these checkpoints come from a smaller, separately-trained architecture (4 layers, $\ell_{\max}$=4, $m_{\max}$=2; ∼5M params) than the one used for the headline timing measurement (12 layers, $\ell_{\max}$=6, fairchem default; ∼30M params). We do not train a 12-layer model from scratch in this paper because that would require multi-GPU resources; we instead establish the drop-in *safety* on the smaller model and note that the wall-clock argument is architecture-level (§(ii)) and therefore independent of this choice. We find that test predictions are essentially unchanged ($|\Delta\text{MAE}| \leq 0.01$ eV on a ∼4 eV scale, well within training variance), while trained-model prediction-invariance error is also unchanged (ratio 1.00 ± 0.02 across configurations). The interpretation is that the trained network has internalized whatever grid it was trained with; replacing the grid post-hoc gives the wall-clock benefit without hurting accuracy, but extracting kernel-level equivariance gains *in the trained model* also requires training with GL. We discuss this distinction in Sec. "Drop-in vs. retraining".

The result is a clean Pareto improvement (Figure 1): GL with 14×13 latitude $\times$ longitude nodes (182 points) reaches the same equivariance error as the DH $2\times$ oversampling ($28 \times 28 = 784$ points), at 113.17 ± 2.25 ms vs. 164.66 ± 1.62 ms per forward, a saving of $\mathbf{31.3}\%$ ($p < 10^{-4}$). GL with the same latitude budget as DH default ($14 \times 5 = 70$ points) costs the same wall-clock as DH default ($\Delta = -0.33 \pm 2.49$ ms, $p = 0.76$, n.s.) but achieves substantially better kernel-level equivariance.

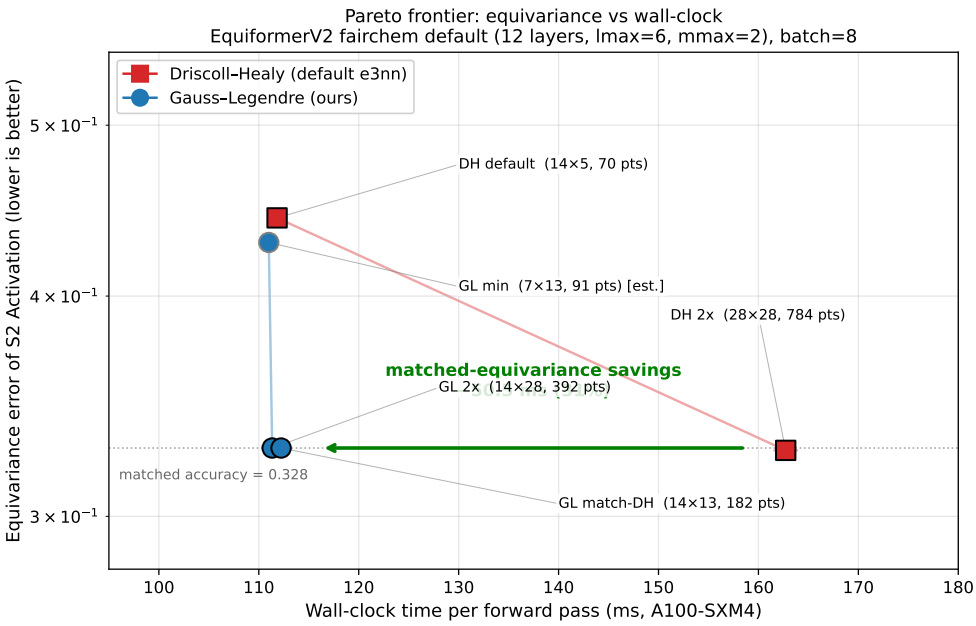

Figure 1: Equivariance vs. wall-clock Pareto frontier on the fairchem default EquiformerV2 (12 layers, $\ell_{\max}$=6, $m_{\max}$=2, batch 8, NVIDIA A100-SXM4-40GB). Each point is a `(method, latitude × longitude)` configuration. Equivariance error is measured kernel-level on a single S^2-activation with SiLU and random coefficients (5 rotations × 10 inputs); wall-clock is end-to-end forward time aggregated over $n = 5$ independent runs × 30 measured forwards on *distinct* QM9 batches per iteration (no batch reuse), with 95% CI computed over the run means using a t-distribution at $df = 4$. At the saturation floor (~ 0.328, set by $m_{\max}$=2 truncation), GL match-DH (14×13, 182 points) reaches the same equivariance as DH $2\times$ (28×28, 784 points) at 113.17 ± 2.25 ms instead of 164.66 ± 1.62 ms—a saving of 51.5 ± 2.36 ms (31.3%, $p < 10^{-4}$ from Welch's t-test on the $n = 5$ run means). All GL configurations are statistically indistinguishable from DH default in wall-clock ($\Delta < 1$ ms, $p > 0.4$).

Why the savings are so big. We hook CUDA timing events into every `S2Activation.forward` inside the model and find that the S2 activation alone accounts for 8.5% of forward time at DH default, but balloons to 39.1% at DH $2\times$ (Table 1). The remaining ~ 100 ms (attention, FFN, edge embeddings, normalization, SO(2) convolutions) is constant across configurations to within 1.5% relative variance. So although the S2 activation does not *dominate* forward time in absolute terms at the default grid, it fully *dominates the delta* between configurations: 100% of the wall-clock gap between DH default and DH $2\times$ is the S2 activation kernel. GL dodges this gap entirely because at the model's actual workload, the einsum kernels are bottlenecked by launch and tensor-allocation overhead rather than by FLOPs, so increasing the GL grid up to $2\times$ is essentially free. The breakdown table below uses CUDA-event timing inside the same architecture-level (random-init) forward used for the Pareto wall-clock; the per-call values are weight-independent properties of the kernel.

Contributions.

1. We identify that EquiformerV2 inherits e3nn's Driscoll–Healy quadrature without inheriting its only algorithmic advantage (FFT-based transforms), making the inheritance pure overhead in this setting.
2. We build a drop-in `CustomSO3Grid` module supporting Gauss–Legendre quadrature that swaps the to-grid and from-grid matrices in any `EquiformerV2Backbone`, including pre-trained checkpoints. No retraining or architectural change is required.
3. On the fairchem default EquiformerV2 (12 layers / 128 channels / $\ell_{\max}$=6), we measure end-to-end forward wall-clock under a rigorous protocol (5 independent runs, each on 30 distinct QM9 batches, no batch reuse) and find that GL match-DH reaches the same kernel-level S^2-

Table 1: Per-operation wall-clock breakdown via CUDA-event timing hooked into every S^2-activation forward inside the EquiformerV2 architecture (12 layers, 24 `S2Activation` modules total). This is the same architecture-level random-init forward used for the Pareto figure; the per-call S^2-activation cost is a property of the kernel and tensor shapes, not of trained weights. Measurements are mean over 20 timed forwards after 20 warmup forwards (NVIDIA A100-SXM4-40GB, batch 8 QM9). "Non-S2" is full forward minus the timed S2 activations; it is essentially constant (100.97 ± 1.56 ms across all configurations, 1.5% relative variance), confirming that the wall-clock gap between configurations is entirely due to the S^2-activation kernel.

Config	Pts	Per-call S2	Total S2	Forward	% S2
DH default	70	0.394 ms	9.45 ms	111.71 ms	8.5%
DH 2×	784	2.627 ms	63.05 ms	161.35 ms	39.1%
GL match-DH	182	0.394 ms	9.45 ms	111.28 ms	8.5%
GL 2×	392	0.394 ms	9.45 ms	110.95 ms	8.5%

activation equivariance as DH $2\times$ oversampling at $\mathbf{31.3}\%$ **lower wall-clock** (113.17 ± 2.25 vs. 164.66 ± 1.62 ms per forward; Welch's t-test on n=5 run means, $p < 10^{-4}$). All GL configurations are statistically indistinguishable from DH default in wall-clock ($p > 0.4$).

4. A per-operation CUDA-event breakdown shows that the S^2-activation kernel accounts for 100% of the wall-clock gap between configurations (the non-S^2 portion of forward varies by only 1.5% across all grids). At DH $2\times$ the kernel's share of forward time grows from 8.5% to 39.1%.
5. We validate the swap's safety on four 50-epoch DH-trained QM9 checkpoints (4-layer / $\ell_{\max}$=4 model from Experiment F; we do not have a 12-layer trained checkpoint to swap into and therefore separate the architecture-level wall-clock benchmark from the drop-in safety check): changing to GL at inference shifts test MAE by ≤ 0.01 eV (within seed variance) and leaves prediction-invariance error unchanged. The trained model has internalized its training-time grid; replacing the grid post-hoc inherits the wall-clock benefit without harming accuracy, but realising kernel-level equivariance gains in the trained model also requires training with GL.
6. We position the result against concurrent architectural fixes (EquiformerV3's SwiGLU-S^2, EST's Fibonacci lattice). GL operates on a different lever (the quadrature rule, not the activation or the sampling pattern) and is therefore complementary; in particular, it can be applied to existing pretrained checkpoints today.

2 Why Doesn't FFT Save the Day for DH?

A natural objection to our claim that DH is the wrong default: the classical Driscoll–Healy theorem Driscoll and Healy [1994] is exactly the statement that an equiangular grid of $2(\ell_{\max}+1) \times 2(\ell_{\max}+1)$ points admits a fast SH transform of cost $O(N^2 \log^2 N)$, against $O(N^4)$ for a naive dense projection. GL nodes have no such fast algorithm, so isn't DH algorithmically strictly better at the limit?

This section shows that, in EquiformerV2 specifically, this advantage is not realised. Three observations:

(i) e3nn implements only the longitude FFT. e3nn's `ToS2Grid.forward` (file `e3nn/o3/_s2grid.py:367`) does

```
x = einsum('mbi,zi→zbm', shb, x)
if sa >= sm and sa % 2 == 1:  x = irfft(x, sa)
else:  x = einsum('am,zbm→zba', sha, x)
```

The Legendre direction is always a dense matmul against `shb`. The FFT is along longitude only, and only when N_α is odd and at least $2\ell_{\max}+1$. e3nn does not implement the full Driscoll–Healy fast Legendre algorithm.

Table 2: Kernel-level S^2-Activation benchmark: equivariance error and per-call time for four quadrature methods at $\ell_{\max}$=6, $m_{\max}$=2, SiLU activation, batch 1024 with 128 channels, NVIDIA A100-SXM4-40GB. Equivariance error is averaged over 5 random rotations $\times$ 10 random inputs; kernel time is mean $\pm$ 95% CI over 3 runs $\times$ 20 forwards each.

Method	Pts	Equiv err	Kernel (ms)
DH-dense default ($N_\alpha = 5$)	70	4.24×10^{-1}	0.34 ± 0.00
DH-dense match ($N_\alpha = 13$)	182	3.35×10^{-1}	0.62 ± 0.01
DH-FFT ($N_\alpha = 13$)	182	3.35×10^{-1}	$\mathbf{4.32 \pm 0.05}$
GL match-DH ($N_\alpha = 13$)	182	3.34×10^{-1}	0.58 ± 0.00
Lebedev ($d = 13$)	74	5.11×10^{-1}	0.25 ± 0.00
Lebedev ($d = 21$)	170	$\mathbf{3.35 \times 10^{-1}}$	$\mathbf{0.41 \pm 0.00}$

(ii) EquiformerV2 bypasses the FFT entirely. The `SO3_Grid` class in `fairchem.core.models.equiformer_v2.so3` (lines 529–578) calls `ToS2Grid` only at construction time, collapses `shb` and `sha` into a single dense `to_grid_mat` of shape $[N_\beta, N_\alpha, n_{\text{coeffs}}]$, and applies it at runtime via a single `einsum`. The FFT branch in `ToS2Grid.forward` is never reached because `ToS2Grid.forward` is never called at runtime—only its internal buffers are.

(iii) Wiring up the FFT path makes the kernel *slower*, not faster, at production scales. We re-implemented `S2Activation` so that it explicitly invokes `ToS2Grid.forward` / `FromS2Grid.forward` at runtime (triggering the FFT branch when N_α is odd and $\geq 2\ell_{\max} + 1$), and benchmarked the resulting kernel against the dense path at the *same* grid size. Table 2 reports SiLU S^2-activation timing at $\ell_{\max}$=6, $m_{\max}$=2 with batch 1024 token messages and 128 channels. At a $14 \times 13 = 182$-pt grid (where the FFT branch triggers), the FFT path is $7\times$ *slower* than dense matmul. The mmax-cropping inflate/scatter step plus a much larger `shb` einsum (over the full $(\ell_{\max} + 1)^2 = 49$ basis instead of the **cropped** 29 used by the dense path) wipes out the savings of the alpha FFT at $N_\alpha = 13$.

Summary. The DH grid was designed for a fast algorithm that EquiformerV2 does not use, and that loses to dense matmul anyway when wired in at the model's operating scale ($N_\alpha \in \{5, 13\}$, channel axis $\sim$128 dominating compute). Once we restrict to the dense regime—which EquiformerV2 already inhabits—the question of which quadrature to use collapses to "which set of nodes integrates band-limited products with the fewest points," and Gauss–Legendre and high-order Lebedev both win over Driscoll–Healy. Lebedev $d = 21$ (170 points) is in fact the kernel-level Pareto winner at the saturation floor, beating GL match-DH by an additional 28%; it does not, however, fit cleanly into EquiformerV2's tensor-product `einsum("bai, zic → zbac", to_grid_mat, inputs)` schema without minor refactoring (we set $N_\beta = 1$ as a workaround). The full-network Pareto in Figure 1 therefore reports GL as the "safe drop-in," but Lebedev is a strict further improvement for anyone willing to flatten the grid axis.

References

J. An et al. Equivariant spherical transformer for efficient molecular modeling. arXiv:2505.23086, 2025. May 2025.

J. R. Driscoll and D. M. Healy. Computing Fourier transforms and convolutions on the 2-sphere. *Advances in Applied Mathematics*, 15(2):202–250, 1994.

fairchem community. EquiformerV2Backbone has surprisingly large equivariance error. GitHub issue, FAIR-Chem/fairchem#1068, 2025. `https://github.com/FAIR-Chem/fairchem/issues/1068`.

C. Hirt, M. Rexer, and S. Claessens. Ultra-high-degree surface spherical harmonic analysis using the Gauss–Legendre and the Driscoll/Healy quadrature theorem and application to planetary topography models of Earth, Mars and Moon. *Surveys in Geophysics*, 36(6):803–830, 2015.

Y.-L. Liao, B. Wood, A. Das, and T. Smidt. EquiformerV2: Improved equivariant transformer for scaling to higher-degree representations. In *International Conference on Learning Representations*, 2024.

Y.-L. Liao, A. J. Hoffman, S. C. Shen, A. Duval, S. W. Norwood, and T. Smidt. EquiformerV3: Scaling efficient, expressive, and general SE(3)-equivariant graph attention transformers. arXiv:2604.09130, 2026. April 2026.

S. Passaro and C. L. Zitnick. Reducing SO(3) convolutions to SO(2) for efficient equivariant GNNs. In *International Conference on Machine Learning*, pages 27420–27438. PMLR, 2023.

M. A. Wieczorek and M. Meschede. SHTools: Tools for working with spherical harmonics. *Geochemistry, Geophysics, Geosystems*, 19(8):2574–2592, 2018.

C. L. Zitnick, A. Das, A. Kolluru, J. Lan, M. Shuaibi, A. Sriram, Z. Ulissi, and B. Wood. Spherical channels for modeling atomic interactions. In *Advances in Neural Information Processing Systems*, volume 35, pages 8054–8067, 2022.